Lecture 2: Finish Linear Regression, Regularized Regression

Max G'Sell
Machine Learning I: 46-926

November 4, 2020

Announcements

- Current office hour schedule:
  - Monday, 10:20am: Me
  - Tuesday, 5pm: Vaibhav
  - Tuesday, 8pm: Wanshan
  - Thursday, TBD: Me

  How does this look? Feedback welcome.

- First homework has now been turned in! Let's talk about it!

- Homework 2 will come out by tomorrow morning and is due **Wednesday before class**.

- First quiz! Out Friday, due Sunday.
  - Simple, short questions.
  - Timed.
  - Your own work.

Today: Finishing basics, Linear Regression and Regularization

- Discuss the homework
- Last comments on linear regression
- Extending linear regression to work in high dimensions
  - Ridge Regression
  - Lasso

Review: Supervised learning

Observe $(X_1, Y_1), \ldots (X_n, Y_n) \stackrel{iid}{\sim} \mathcal{P}$. Estimate $\hat{\mu}(x)$ so that $\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X))$ is small for new $(X, Y) \sim \mathcal{P}$.

If we are minimizing expected squared prediction error:

- If we actually knew $\mathcal{P}$, the best we could do is

$$\mu(x) = \mathbb{E}(Y|X = x)$$

- If we are estimating $\hat{\mu}$, then

$$\mathbb{E}\left(\mathcal{L}(Y, \hat{\mu}(X)) \mid X = x\right) = \sigma_x^2 + \text{bias}(\hat{\mu}(x))^2 + \text{var}(\hat{\mu}(x))$$

Review: Bias and variance

$$\text{bias}\left(\hat{\mu}(x)\right) = \mathbb{E}\left[\hat{\mu}(X)|X = x\right] - \mu(x)$$

Bias: Error in the expectation of our estimator. Does not depend on the randomness of our particular training data realization, just the flexibility of our function. Making $\hat{\mu}$ more flexible usually decreases bias.

$$\text{var}\left(\hat{\mu}(x)\right) = \mathbb{E}\left[\left(\hat{\mu}(X) - \mathbb{E}\left[\hat{\mu}(X)|X = x\right]\right)^2 \mid X = x\right]$$

Variance: Error from the variance of our estimator around *its* mean. Does not depend on the true function μ! Tends to increase with the flexibility of $\hat{\mu}$, since a change in data has more effect.

Review: Linear regression

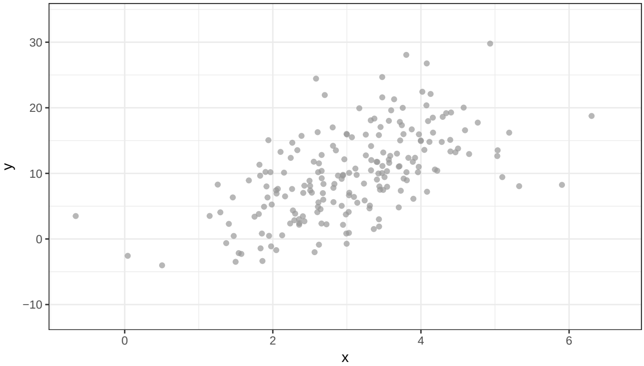
We can think of linear regression as one way to approximate $\mathbb{E}(Y|X = x)$ across x values, imagining:

$$\mathbb{E}(Y|X = x) \approx x^T\beta$$

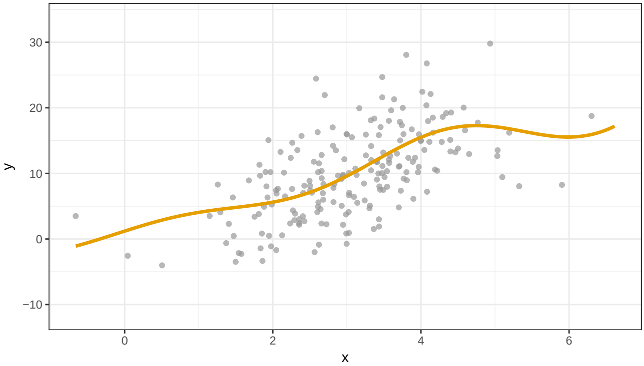
This can work well if the linear model is not a bad approximation.

It has the advantage of being very simple, which can behave better than very flexible models in high-noise settings.

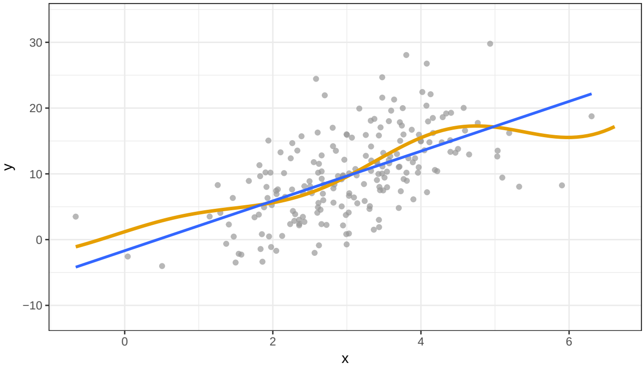
Regression function demo



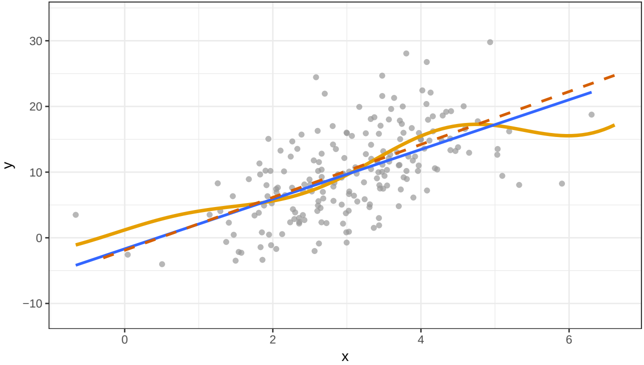
Regression function demo



Regression function demo



Regression function demo



Linear regression

$$\begin{aligned}\hat{\mu}(x) &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

If the true $\mathbb{E}(Y|X)$ is very far from linear, this model will be highly *biased*, since even $\mathbb{E}\hat{\mu}(X)$ will never be able to match $\mathbb{E}(Y|X)$.

Even with infinite data, the best linear model would still be terrible. You learned (and we will discuss) much more flexible models to avoid this problem.

We will have other problems even when the linear model is correct!

Linear regression when the linear model is true!

Ok, so suppose that we use linear regression just to make predictions. Furthermore, suppose that the true conditional mean is linear! What will happen?

We know that least squares has nice properties when the linear model is true – e.g., it is unbiased and has minimum variance among unbiased, linear estimates (Gauss-Markov Theorem).

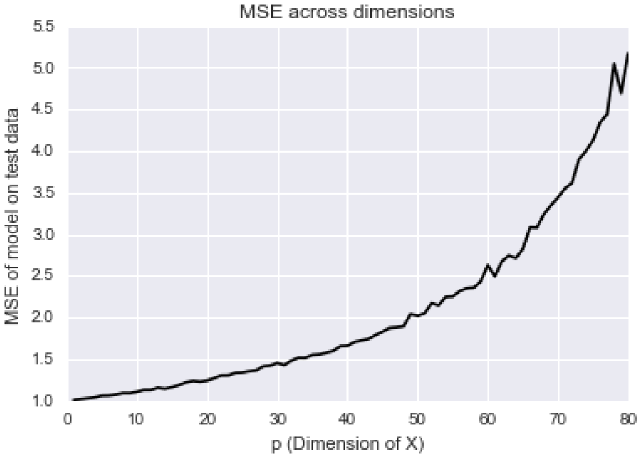
However...

Linear regression in high dimensions

What happens when we have a bunch of variables? (Big data!)

However, we noticed right at the end of class that as the number of variables, p, grows large, the variance of linear regression grows rapidly. This leads to poor predictions even with the linear model is true!

Linear regression in high dimensions



This is a simulation from a true linear model with independent Gaussian errors – the dream! (Also the plot from your homework.)

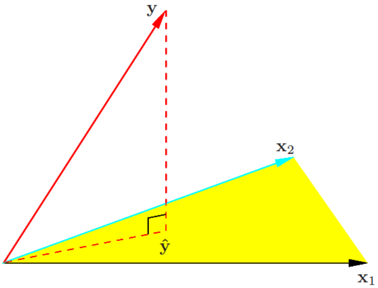
Linear regression in high dimensions

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

As dimension increases, the prediction error increases (super-)linearly! When dimension is bigger than the number of observations,

What is going on, and how can we fix it?

Two Pictures: n-space



$$\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Views the columns $(x_{1j}, \ldots, x_{nj})$ as vectors in $\mathbb{R}^n$, along with $(y_1, \ldots, y_n)$ and $(\varepsilon_1, \ldots, \varepsilon_n)$

$$Y = \sum_{i=1}^{n} X_j \beta_j$$

Easy to see:

- How to fit our model
- What happens when new variables are added?
- How the error ε interacts with dimension?

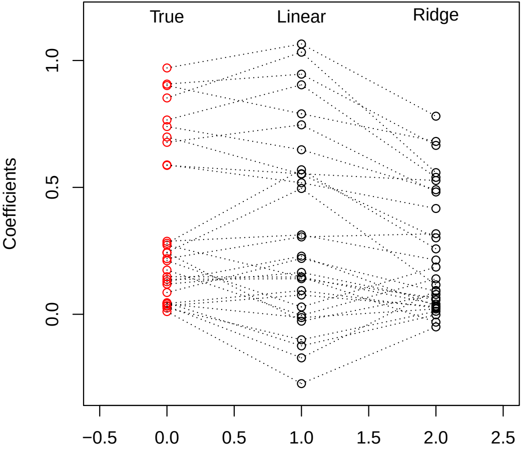
Fixing linear regression in high dimensions

How do we reduce this error?

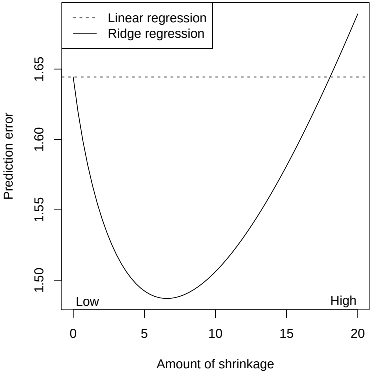
$$\mathbb{E}\left((Y - \hat{\mu}(X))^2\right) = \sigma_x^2 + \mathrm{Bias}\left(\hat{\mu}(X)\right)^2 + \mathrm{Var}\left(\hat{\mu}(X)\right)$$

Example: visual representation of ridge coefficients

A visual representation of the ridge regression coefficients for the same example ($n = 50$, $p = 30$, and $\sigma^2 = 1$; 10 large true coefficients, 20 small) at $\lambda = 25$:



Does it work?



Linear regression:
Squared bias $= 0$
Variance ≈ 0.633
Pred. error $\approx 1 + 0.633$
≈ 1.633

Ridge regression, at its best:
Squared bias ≈ 0.077
Variance ≈ 0.403
Pred. error $\approx 1 + 0.077 + 0.403$
≈ 1.48

This is upsetting! And amazing!

Ridge regression

Ridge regression is like least squares but shrinks the estimated coefficients towards zero. Given a response vector $y \in \mathbb{R}^n$ and a predictor matrix $X \in \mathbb{R}^{n \times p}$, the ridge regression coefficients are defined as

$$\begin{aligned}\hat{\beta}^{\text{ridge}} &= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \\ &= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_2^2}_{\text{Penalty}}\end{aligned}$$

Ridge regression

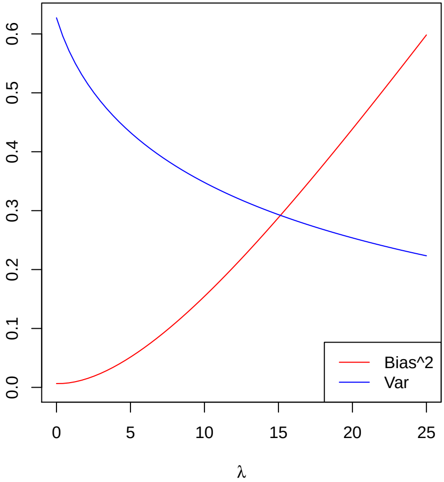
$$\hat{\beta}^{\text{ridge}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$
$$= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \underbrace{\|y - X\beta\|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_2^2}_{\text{Penalty}}$$

Here $\lambda \geq 0$ is a tuning parameter, which controls the strength of the penalty term. Note that:

- When $\lambda = 0$,
- When $\lambda = \infty$,
- For λ in between, we are balancing two ideas: fitting a linear model of y on X, and shrinking the coefficients

Example: bias and variance of ridge regression

Bias and variance for our last example ($n = 50$, $p = 30$, $\sigma^2 = 1$; 10 large true coefficients, 20 small):



Important details: centering

When including an intercept term in the regression, we usually leave this coefficient unpenalized.

Otherwise we could add some constant amount c to the vector y, and this would not result in the same solution.

Hence ridge regression with intercept solves

$$\hat{\beta}_0, \hat{\beta}^{\text{ridge}} = \underset{\beta_0 \in \mathbb{R},\, \beta \in \mathbb{R}^p}{\operatorname{argmin}} \; \|y - \beta_0 \mathbf{1} - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

If we center the columns of X, then the intercept estimate ends up just being $\hat{\beta}_0 = \bar{y}$, so we usually just assume that y, X have been centered and don't include an intercept

Important details: scaling

Also, the penalty term $\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$ is unfair if the predictor variables are not on the same scale. (Why?)

Therefore, if we know that the variables are not measured in the same units, we typically scale the columns of X (to have sample variance 1), and then we perform ridge regression

We see that squishing the coefficients toward zero gives:

- A decrease in variance!
- A decrease in MSE!
- An increase in bias...

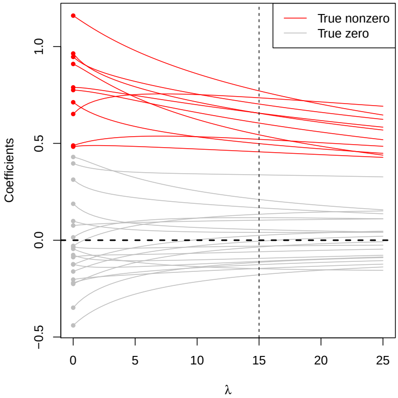
However, ridge regression:

- Seems quite harsh on large, important coefficients.

- Leaves us with a large linear model, which is not that interpretable when we have many variables.

Ridge regression coefficients

Ridge regression coefficients, plotted against λ:



The red paths correspond to the true nonzero coefficients; the gray paths correspond to true zeros. The vertical dashed line at $\lambda = 15$ marks the point above which ridge regression's MSE starts losing to that of linear regression

An important thing to notice is that the gray coefficient paths are not exactly zero; they are shrunken, but still nonzero

The lasso

The lasso[1] estimate is defined as

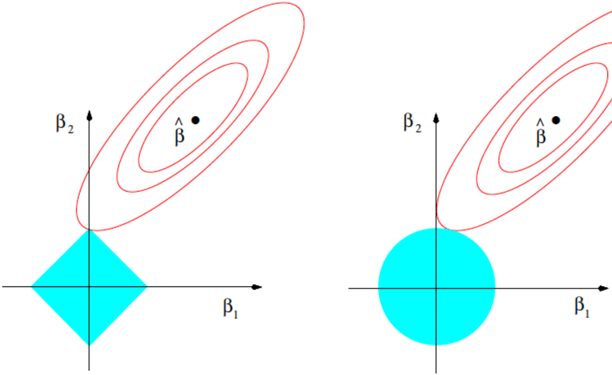
$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \; \|y - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

$$= \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \; \underbrace{\|y - X\beta|_2^2}_{\text{Loss}} + \lambda \underbrace{\|\beta\|_1}_{\text{Penalty}}$$

The squared ℓ_2 penalty $\|\beta\|_2^2$ of ridge regression, has been replaced by an ℓ_1 penalty $\|\beta\|_1$. Even though these problems look similar, their solutions behave very differently

Note the name "lasso" is actually an acronym for: Least Absolute Selection and Shrinkage Operator

[1] Tibshirani (1996), "Regression Shrinkage and Selection via the Lasso"



β_2
$\hat{\beta}$
β_1
β_2
$\hat{\beta}$
β_1

$$\hat{\beta}^{\text{lasso}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \; \|y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

The tuning parameter λ controls the strength of the penalty.

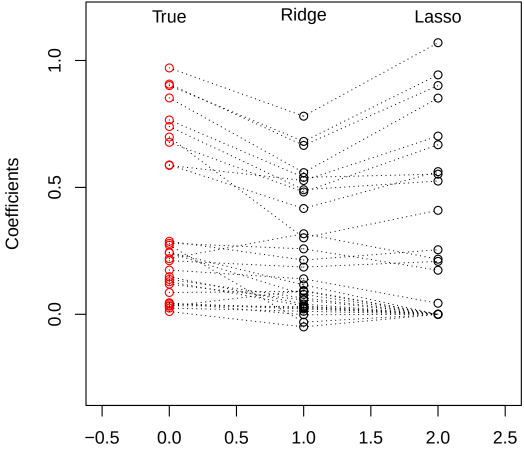
- If $\lambda = 0$:
- As $\lambda \to \infty$:

For λ in between these two extremes, we are balancing two ideas: fitting a linear model of y on X, and shrinking the coefficients. But the nature of the ℓ_1 penalty causes some coefficients to be shrunken to zero exactly

This leads to variable selection!

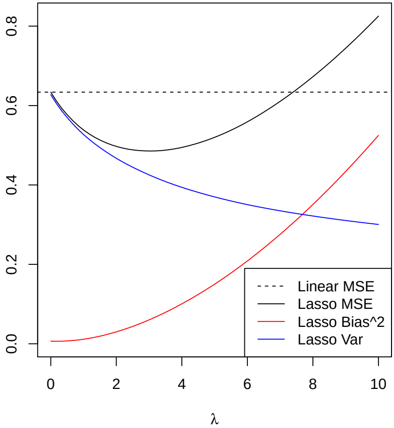
Example: visual representation of lasso coefficients

Our running example from last time: $n = 50$, $p = 30$, $\sigma^2 = 1$, 10 large true coefficients, 20 small. Here is a visual representation of lasso vs. ridge coefficients (with the same degrees of freedom):



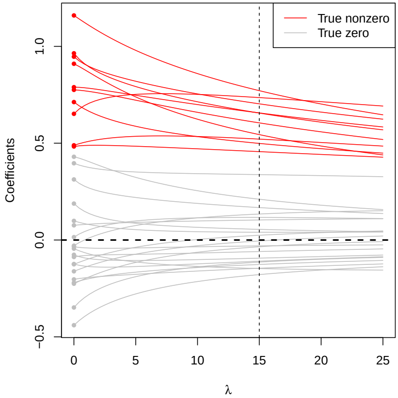
Example: subset of small coefficients

Example: $n = 50$, $p = 30$; true coefficients: 10 large, 20 small



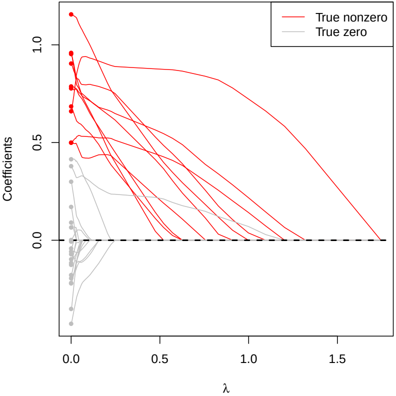
Remember this behavior for Ridge Regression?

Ridge regression coefficients, plotted against λ:



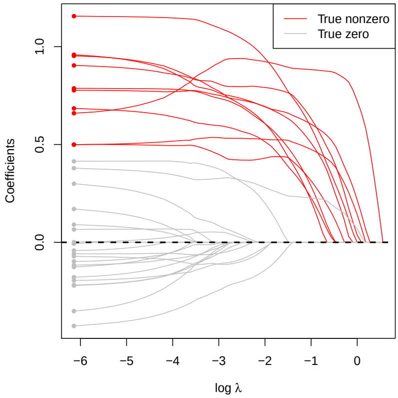
What changes?

Lasso coefficient paths



In the lasso, we get exact zeros! Moreover, the large coefficients are not impacted nearly as much!

Lasso coefficient paths



In the lasso, we get exact zeros! Moreover, the large coefficients are not impacted nearly as much!

It is often easier to view lasso results on the $\log \lambda$ scale.

Advantages of sparsity

- Interpretability: We can understand what the model relies on for prediction (understanding $\hat{f}$)
- We might gain some insight into the underlying data (though not causally) (helping to understand f)
- If we're building a predictive score, we can measure fewer things in the future (simpler $\hat{f}$ to apply later)

Sidenote: Can we do inference (e.g., p-values, confidence intervals, etc)? Not easily!

Important details

Just like in ridge regression:

- We don't penalize our intercept, if one is included. Again, columns are often centered and then intercept omitted.

- The penalty term $\|\beta\|_1 = \sum_{j=1}^{p} |\beta_j|$ is not fair if the predictor variables are not on the same scale. Hence, we often scale the columns of X to have variance 1.

How is it actually solved?

[Reminder: Show software demos]

Interview question

Suppose that you regress Y on X_1, X_2 using both ridge regression and the lasso (with $\lambda > 0$). If $X_1 = X_2$, what does:

The solution to ridge look like

The solution to lasso look like

Simple case: Orthogonal X

Consider the very simple case where $X = I$. We can examine how both ridge regression and the lasso behave on this data. This is easy, because the variables do not interact with one another, but it gives an intuition for their more general behavior.

When $X = I$, note that the RSS becomes

$$\|y - X\beta\|_2^2 = \|y - \beta\|_2^2 = \sum_{i=1}^{n} (y_i - \beta_i)^2.$$

Simple case: Orthogonal X

The corresponding penalized forms then become

Ridge:

$$\hat{\beta}_{\text{ridge}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - \beta_i)^2 + \lambda \beta_i^2$$

Lasso:

$$\hat{\beta}_{\text{lasso}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - \beta_i)^2 + \lambda |\beta_i|$$

We can minimize separately for each i!

For ridge regression, differentiating yields

and therefore

$$\hat{\beta}_i =$$

Thus the estimates from ridge regression correspond to the least squares estimates reduced by a constant multiple.

Note that this can never give zero coefficients, and that it penalizes large coefficients quite a bit.

For the lasso, it turns out that the minimizer of

$$(y_i - \beta_i)^2 + \lambda|\beta_i|$$

is given by

$$\hat{\beta}_i =$$

Thus estimates from ridge regression correspond to shrinking the least squares estimates toward zero by an additive constant (without crossing zero).

This can give zero coefficients, and is also relatively harsher on small coefficients than larger ones. It appears to be better suited to sparse settings.

Regularized regression

Least squares solves the problem:

$$\underset{\beta}{\operatorname{argmin}} \|y - X\beta\|_2^2$$

In regularized regression, we add a penalty to this squared error loss term

$$\underset{\beta}{\operatorname{argmin}} \|y - X\beta\|_2^2 + P_\lambda(\beta)$$

You've seen several examples: $P_\lambda(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_\lambda(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both "pull" β toward 0, causing it to be less variable. However, by making β smaller than the least squares estimate, they add bias.

The λ parameter determines how strong this penalty is. A larger λ leads to a higher bias and a smaller variance.

Regularized regression

$$\underset{\beta}{\operatorname{argmin}} \|y - X\beta\|_2^2 + P_\lambda(\beta)$$

$P_\lambda(\beta) = \lambda \sum_j \beta_j^2$ (ridge regression) and $P_\lambda(\beta) = \lambda \sum_j |\beta_j|$ (lasso).

These penalties both "pull" β toward 0, causing it to be less

However, by making β smaller than the least squares estimate, they add

The λ parameter determines how strong this penalty is. A larger λ leads to a higher and a smaller

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

We will see at least:

- Logistic regression with a lasso penalty
- The elastic net

Extensions of the Lasso

Adding an L_1 penalty can induce sparsity in a variety of settings.

An entire industry has been developed around producing related methods and related theory.

Other classic examples:

- The group lasso: setting groups of variables to zero
- Graphical lasso: estimates networks of dependence between variables
- Fused lasso: estimates piecewise-constant functions
- Trend filtering: estimates piecewise-polynomial functions
- Low-rank matrix decomposition: estimate a low-rank approximation to a matrix
- Sparse additive models
- Vector auto-regressive models